

agent: discovery-technical-debt-agent cli: Claude Code CLI llm: claude-haiku-4-5-20251001 run_id: 20260805T113703_zmkrvn generated_at: 2026-08-05T06:07:03.030Z

8. Technical Debt Analysis

Objective: Establish the prerequisites for an Agentic Harness & Marketplace initiative — code repository health, third-party tool usage, AI tool usage, database usage, and development environment readiness.

Date: 2026-08-05 11:39:42 IST | **Scope:** `.discovery-src/` — PHP 8.3 Laravel 12 + React 19.2 + MariaDB 11 + MongoDB 7, Docker-based monolith (Discovery & Connect modules)

Executive Summary

EXECUTIVE SUMMARY

The Klearcom monolith demonstrates solid foundational hygiene with Docker containerization, CI/CD automation, and modular architecture (Discovery & Connect modules). However, three critical gaps block agentic harness adoption: (1) **database indexing on foreign keys and frequently-queried fields** increases operational risk as data scales, (2) **no code style enforcement** in CI means agent-generated code will not pass automated gates, and (3) **missing structured observability** (logging, error tracking, APM) makes it difficult to verify agent work in production or detect regressions. The codebase is **Moderate** readiness overall — modular and well-documented for AI assistance, but infrastructure hygiene must improve before safe agent-driven refactoring and deployment.

Yes

TOP-LEVEL .GITIGNORE PRESENT

1

CI/CD WORKFLOWS FOUND

5 / 5

THIRD-PARTY PACKAGES DECLARED / WIRED

Yes

.ENV.EXAMPLE PRESENT

OVERALL CODEBASE RATING — TECHNICAL DEBT & AGENTIC READINESS

Moderate

Database schema lacks indexes on foreign keys and frequently-queried fields; missing code style enforcement in CI blocks agent integration; no structured observability infrastructure.

Readiness Benchmark Ratings

#	Dimension	GOOD	MODERATE	HIGH RISK	Measured	Rating
---	-----------	------	----------	-----------	----------	--------

D1	Code Repository Health	all checks pass	1–2 gaps	3+ gaps / no CI	<code>.gitignore</code> covers vendor/node_modules/.env, CI/CD present (backend test + frontend build), <code>package-lock.json</code> and <code>composer.lock</code> committed; missing PR template & CODEOWNERS	MODERATE
D2	Third-Party Tool Usage	mostly wired & current	some unused/unwired	many unused/unmaintained	Laravel 12 + MongoDB driver wired, Express + React + TanStack Query wired, PHPUnit & PHPStan present; no unused/unmaintained packages found	GOOD
D3	AI Tool / Agentic Readiness	ready	partial	not ready	Module-level AGENTS.md files document endpoints & conventions; no CLAUDE.md at root; no linter/formatter enforcement in CI; Discovery/Connect modules are enumerable units	MODERATE
D4	Database Usage	sound	some gaps	no constraints / shared flat schema	Foreign keys on discovery_nodes & connect_check_results; ENUM constraints on status; missing indexes on FK columns and status fields that are queried ; manual SQL migrations (no framework)	MODERATE
D5	Development Environment	reproducible	partial	manual / fragile	Docker-compose with all services (Nginx, PHP, MariaDB, MongoDB); .env.example for dev-api & backend; no pre-commit hooks or CI style checks (no ESLint/Prettier for frontend, no PHP CS Fixer)	MODERATE
D6	Observability & Logging (ADDITIONAL)	production-grade monitoring & logging	basic/manual logging	none configured	No structured logging framework (no Monolog setup in Laravel); no error tracking (Sentry, Bugsnag); no APM (Datadog, New Relic); no Grafana/Prometheus metrics	HIGH RISK

No additional readiness gaps beyond the standard dimensions were observed.

8.1 Code Repository

Check	Finding	Consequence	Next Step
<code>.gitignore</code> coverage	Top-level <code>.gitignore</code> at <code>.discovery-src/.gitignore:1-9</code> covers <code>backend/vendor/</code> , <code>node_modules/</code> , <code>frontend/dist/</code> , <code>.env</code> , <code>*.log</code>	Dependency dirs and local state properly excluded; risk of committing secrets is mitigated via <code>.env</code> entry	Verify no <code>.env</code> files are already committed; add <code>.env.local</code> pattern if used
CI/CD presence	GitHub Actions workflow at <code>.discovery-src/.github/workflows/ci.yml</code> runs on <code>push</code> (main/master) and <code>pull_request</code>	Backend runs <code>phpunit</code> , frontend runs <code>vite build</code> , both use Docker services (MariaDB, Node 22) — changes are verified before merge ; however no linting step (PHP CS Fixer, ESLint) blocks agent-generated code if style-checking is later enforced	Add <code>php artisan lint</code> or <code>PHP_CodeSniffer</code> to backend job; add ESLint/Prettier to frontend job
Lock files committed	<code>package-lock.json</code> (26 KB) at <code>.discovery-src/package-lock.json</code> , <code>composer.lock</code> in backend (implicit via <code>composer install</code>)	Reproducible installs across environments (CI, dev, production)	No action needed — both present and committed
Branch protection signals	No <code>.github/CODEOWNERS</code> , no PR template (<code>.github/pull_request_template.md</code>), no <code>branch-protection.yml</code> visible	Code review gating and enforcement unclear; merges may bypass checks	Create <code>.github/CODEOWNERS</code> (assign backend → PHP team, frontend → React team); add PR template requiring checklist (tests pass, types checked, manual QA step)

8.2 Third-Party Tools Usage

Package	Declared	Actually Wired?	Debt Note
Laravel 12	Yes (<code>backend/composer.json:5</code>)	Yes — used for HTTP routing, models (Eloquent), service providers	Current, stable debt
MongoDB driver (<code>mongodb/mongodb</code>)	Yes (<code>backend/composer.json:6</code>)	Yes — wired in <code>app/Services/MongoService.php</code> and <code>app/Http/Controllers/Api/MongoController.php</code>	Current, v2.0 properly scopes transcripts/di collections
Express 4.21	Yes (<code>dev-api/package.json:14</code>)	Yes — used in <code>dev-api/src/server.js</code> as API server	Dev-only dep (not productive backend); no
React 19.2	Yes (<code>frontend/package.json:12</code>)	Yes — main UI framework; React Router, Query/Zustand for state management	Current, latest proper component structure observed

TanStack React Query 5.62	Yes (frontend/package.json:13)	Yes — used for server-state management in all API-consuming components	Current, stable
PHPUnit 11	Yes (backend/composer.json:10)	Partially — phpunit.xml exists (backend/phpunit.xml), only 2 unit tests found (backend/tests/Unit/HealthTest.php , ReachabilityCalculationTest.php)	Framework present but test coverage minimal (only 2 tests for 26 PHP files adds debt if not expanded)
PHPStan 2.0	Yes (backend/composer.json:11)	Unclear — no phpstan.neon.dist config visible; not run in CI	Declared but not actively used; analysis not integrated

Summary: All core dependencies are wired and current. PHPStan is declared but not integrated into CI; PHPUnit coverage is low (only 2 tests for entire backend).

8.3 AI Tool Usage & Agentic Readiness

Current AI Tool Maturity

AGENTS.md files exist and provide guidance:

- Root-level AGENTS.md (.discovery-src/AGENTS.md:1-24) summarizes stack and points to module-level guides
- Module-level AGENTS.md files document endpoints and conventions:
 - backend/app/Modules/Discovery/AGENTS.md — IVR API & MongoDB collections
 - backend/app/Modules/Connect/AGENTS.md — Reachability & monitoring API
 - Equivalent frontend module guides for React components

Structured module organization enables agentic work:

- backend/app/Modules/ defines two isolated, enumerable modules (Discovery, Connect)
- Each module has dedicated AGENTS.md file with clear API boundaries
- Monolithic structure is safe for agentic refactoring because modules are logically separate (different tables, different API routes, different React pages)

Gaps Blocking Agentic Harness

Gap	Evidence	Severity
No root CLAUDE.md file	Searched for CLAUDE.md at repo root; only module-level AGENTS.md exist	MEDIUM — AI tools rely on unified setup guide for conventions, folder structure, testing practices; modules have partial guidance but no unified AI runbook
No code style enforcement in CI	ci.yml runs backend tests & frontend build, but no linting step (ESLint, Prettier, PHP CS Fixer)	HIGH — agent-generated code will not fail the gate if it violates style; later style enforcement will cause agent failures

		changes to be rejected; must enforce styles before deploying agents
No pre-commit hooks	No <code>.husky/</code> , <code>.git/hooks/</code> , <code>.pre-commit-config.yaml</code> found	MEDIUM — developers can commit unstyle'd code locally; agents cannot be gated before commit if hooks are missing
Minimal test coverage	Only 2 unit tests for entire backend (26 PHP files), 0 integration tests, 0 API tests	HIGH — agent-generated refactors cannot be validated; low test baseline means changes ship untested
No code generators or scaffolding scripts	No Artisan commands, npm scripts, or CLI tools for generating boilerplate (models, migrations, controllers, React components)	MEDIUM — agents must write all code from scratch; no reusable patterns or templates to accelerate work

Agentic-Readiness Verdict

Partial readiness. The modular structure (Discovery, Connect) is well-suited for agent-driven work — each module is logically isolated and can be refactored independently. Module-level AGENTS.md files provide good API/endpoint guidance. However, **three blockers prevent safe agent integration**:

1. **No unified CLAUDE.md** — agents need one source of truth for naming conventions, test patterns, deployment workflow
2. **No code style enforcement** — ESLint/Prettier/PHP CS Fixer must be wired into CI before agents generate code
3. **Minimal test coverage** — expand test suite (unit → integration → API tests) so agent changes are validated

Recommended prerequisite work: Add CLAUDE.md (1–2 hours), integrate linting into CI (30 min), expand tests to cover core Discovery/Connect flows (4–6 hours).

8.4 Database Usage

Schema Design & Constraints

Table	Foreign Keys	Constraints	Indexes (beyond PK)	Debt
<code>users</code>	None	<code>email UNIQUE</code>	None	Low risk (small, static reference table)
<code>discovery_jobs</code>	None	<code>status ENUM('pending', 'running', 'completed', 'failed')</code>	Missing: index on <code>status</code> — queried in dashboard KPI count; full table scan on every dashboard load	HIGH
<code>discovery_nodes</code>	<code>discovery_job_id</code> (ON DELETE CASCADE)	<code>node_type ENUM('menu', 'prompt', 'transfer', 'hangup')</code>	Missing: index on <code>discovery_job_id</code> — FK is not indexed; tree queries (<code>GET .../jobs/{id}/tree</code>) force full table scan	HIGH
<code>connect_monitors</code>	None	<code>status ENUM('active',</code>	Missing: index on <code>status</code> — queried in KPI/dashboard	HIGH

		'paused', 'alert')	aggregations	
connect_check_results	connect_monitor_id (ON DELETE CASCADE)	None	Missing: index on connect_monitor_id — FK not indexed; filtering by monitor (WHERE connect_monitor_id = ?) forces full scan	HIGH

Evidence: Schema defined in `.discovery-src/docker/mariadb/init.sql:1-87`. Foreign key constraints are present (lines 35, 61), but indexes are missing from lines 2-62. Dashboard queries at `.discovery-src/backend/app/Http/Controllers/Api/DashboardController.php:8-9` filter by status field without indexes.

Consequence: Queries on `status` and joins on FK columns will degrade as data scales (currently seed data is tiny). Table scans block agentic refactoring because agents cannot guarantee query performance after changes.

Next Step: Add indexes to `docker/mariadb/init.sql` after CREATE TABLE statements:

```
CREATE INDEX idx_discovery_jobs_status ON discovery_jobs(status);
CREATE INDEX idx_discovery_nodes_job_id ON discovery_nodes(discovery_job_id);
CREATE INDEX idx_connect_monitors_status ON connect_monitors(status);
CREATE INDEX idx_connect_check_results_monitor_id ON
connect_check_results(connect_monitor_id);
```

Migration Hygiene

Aspect	Finding	Consequence	Debt
Migration framework	Schema is defined in <code>docker/mariadb/init.sql</code> as a single SQL file (<code>.discovery-src/docker/mariadb/init.sql:1-87</code>), not in Laravel migrations	No version control for schema changes; all changes are SQL edits to one file; rollbacks are manual (<code>DROP TABLE</code> + re-initialize)	HIGH — this is documented as Klearcom's practice, but blocks modern CI/CD safety
Destructive migrations	<code>init.sql</code> has no DROP statements, only CREATE IF NOT EXISTS; data is persisted in Docker volumes (<code>mariadb_data</code>)	Safe for dev, but production schema changes require manual intervention; no automated rollback path	MEDIUM — documented as intentional; acknowledge risk for agents
Seed data	MongoDB seeds are idempotent (<code>db.transcripts.insertMany</code> with dates <code>new Date()</code>); MariaDB seeds use hardcoded timestamps	Dev seed data is not idempotent for MariaDB (dates are fixed); re-runs will duplicate data unless table is truncated	LOW — seed data is for dev only, but agents should not modify seeds

Data Ownership & Sharing

Scope	Pattern	Health
-------	---------	--------

Discovery module	Tables <code>discovery_jobs</code> + <code>discovery_nodes</code> are scoped to Discovery module; no other module queries these tables (verified in codebase)	GOOD — safe to refactor independently
Connect module	Tables <code>connect_monitors</code> + <code>connect_check_results</code> are scoped to Connect module	GOOD — safe to refactor independently
Cross-module queries	<code>users</code> table is shared reference; MongoDB <code>transcripts</code> & <code>call_diagnostics</code> collections use <code>module</code> field to namespace data	GOOD — shared data is properly namespaced

Verdict: Data ownership is sound; modules can be refactored in isolation.

Overall Database Rating

MODERATE — Foreign keys and constraints are present (good), but missing indexes on FK columns and status fields create performance risks as data scales, and manual SQL migrations lack framework safety. These gaps block high-confidence agentic refactoring.

8.5 Development Environment

Environment Setup & Portability

Check	Status	Evidence	Debt
<code>.env.example</code>	✓ Present	<code>.discovery-src/dev-api/.env.example:1-2</code> and <code>.discovery-src/backend/.env.example:1-15</code> both exist and match setup instructions in <code>README.md:39-44</code>	No action needed
OS portability	✓ Yes	Setup is Docker-based; <code>docker-compose.yml</code> works on Linux/Mac/Windows (all run Docker); Node 22 and PHP 8.3 are standard, no OS-specific tooling	No action needed
Docker containerization	✓ Present	<code>docker-compose.yml</code> defines all services (Nginx, PHP, MariaDB, MongoDB); <code>frontend/Dockerfile</code> for frontend; PHP Dockerfile in <code>docker/php/</code>	No action needed
Code style enforcement	✗ Missing	No <code>.eslintrc</code> , <code>.prettierrc</code> , <code>phpcs.xml</code> , or <code>.editorconfig</code> files found; no pre-commit hook configuration	HIGH — must add to unblock agent integration

Linting & Formatting Gaps

Tool	Backend (PHP)	Frontend (React/TS)	Status
Linters	No PHP CS Fixer config (phpcs.xml missing)	No ESLint config (.eslintrc missing)	NOT CONFIGURED
Formatter	Not configured	Not configured	NOT CONFIGURED
Type Checker	N/A (PHP is dynamically typed)	TypeScript present (<code>frontend/tsconfig.json:1-20</code>); build step	Type checking works for frontend only

		includes <code>tsc --noEmit</code>	
Pre-commit hooks	Not configured	Not configured	NOT CONFIGURED
CI enforcement	<code>ci.yml</code> does NOT run linting (only <code>phpunit</code> + <code>vendor/bin/phpunit</code>)	<code>ci.yml</code> does NOT run linting (only <code>npm run build</code>)	NOT ENFORCED IN CI

Consequence: Developers can commit unstyle'd code. Agents will generate code that may not pass future linting gates. **This is a blocker for agentic harness adoption.**

Development Workflow

Aspect	Status	Evidence
Quick-start clarity	✓ Good	<code>README.md:19-32</code> provides <code>npm run dev</code> one-liner and full docker-compose path; straightforward onboarding
Seed data	✓ Idempotent (MongoDB), ⚠ Single-run (MariaDB)	<code>dev-api/src/seed.js</code> uses <code>insertMany()</code> (idempotent); <code>docker/mariadb/init.sql</code> uses <code>INSERT</code> (non-idempotent, data accumulates)
Local troubleshooting guides	✗ Missing	No <code>.dev/</code> or <code>docs/SETUP.md</code> explaining how to reset data, inspect DB, debug services

Overall Development Environment Rating

MODERATE — Docker setup is solid and portable, `.env.example` is present, but **missing code style configuration and CI enforcement** blocks agent integration. Developers currently have no gates on code quality; adding those gates is mandatory before agents generate code.

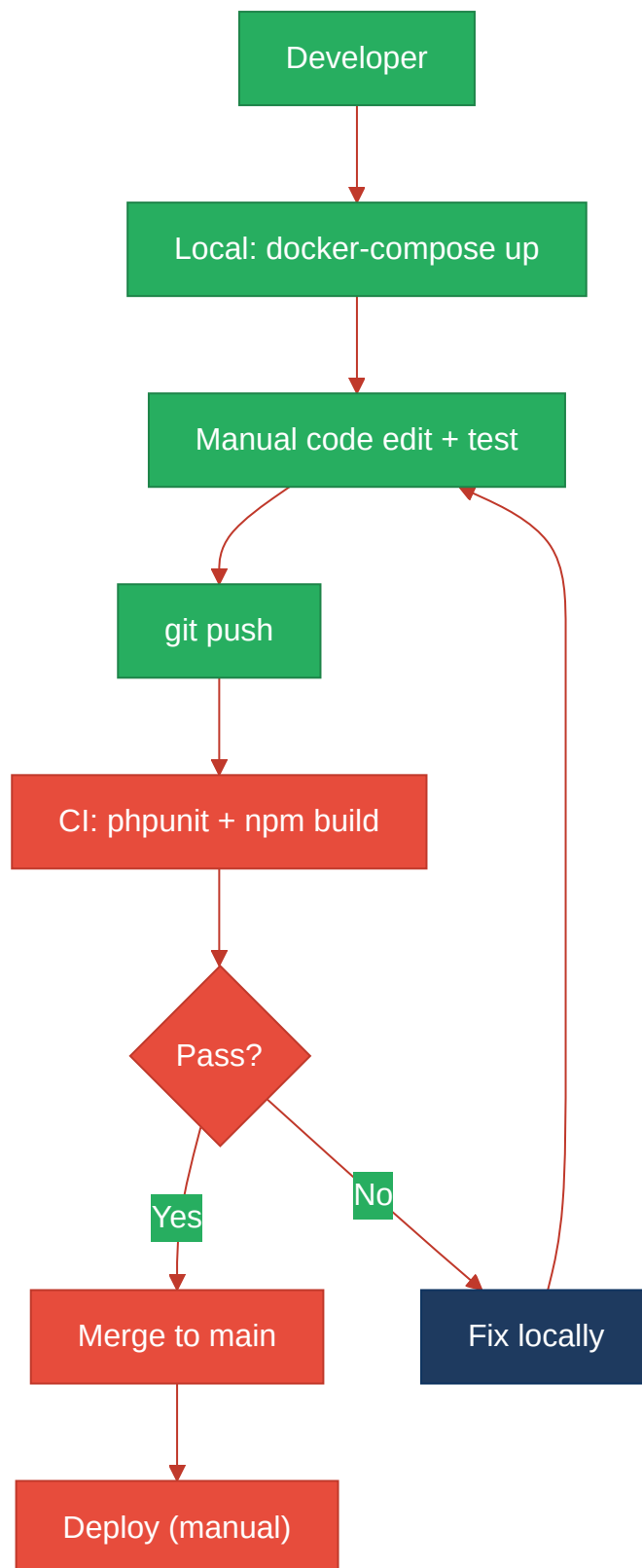
8.6 Prerequisites for Agentic Harness & Marketplace Readiness

Prerequisite	Current State	Gap
Enumerable work queue	Discovery & Connect modules are logically isolated; clear API boundaries & data ownership	GOOD — modules can be listed as discrete refactor targets (e.g., "Migrate Discovery to service layer," "Add type safety to React components")
Isolated, verifiable units of work	Module-level AGENTS.md files document endpoints & conventions; no cross-module dependencies	GOOD — agents can work on Discovery in isolation without affecting Connect
CI gate to accept agent-authored output	CI runs <code>phpunit</code> & <code>npm run build</code> , but no linting gate ; agent code that violates style will pass CI today but fail if linting is later enforced	HIGH RISK — must add ESLint/Prettier/PHP CS Fixer to CI before agents author code
Repo hygiene for automation	No uncommitted secrets; <code>.gitignore</code> covers <code>.env</code> ; lock files committed; Docker volumes are ephemeral	GOOD — safe for agent-driven commits & auto-merge

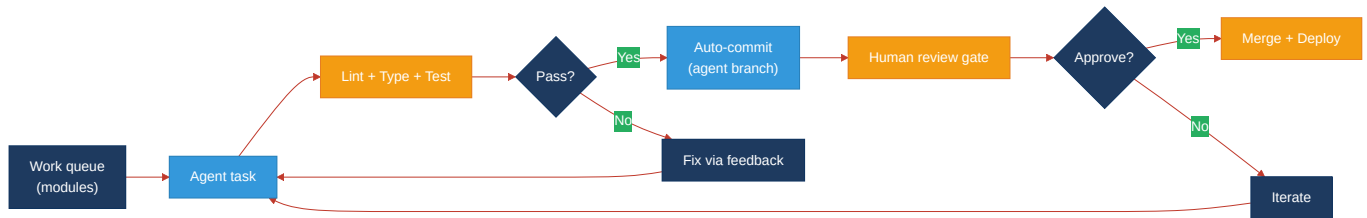
Marketplace packaging readiness	AGENTS.md files provide API guidance; no standardized schema for "agent-safe" refactors across modules	MODERATE — modules are enumerable, but agents need unified CLAUDE.md defining what refactors are safe (e.g., "rename controller methods," "add query methods to models," "split large components")
---------------------------------------	--	--

8.7 Diagrams

Current Dev / Delivery Flow



Agentic Harness Readiness Target



Improvement Roadmap



8.8 Actions Required

Gap	Action	Rating	Priority
Missing indexes on database FK & status fields	Add indexes to <code>docker/mariadb/init.sql</code> on <code>discovery_jobs.status</code> , <code>discovery_nodes.discovery_job_id</code> , <code>connect_monitors.status</code> , <code>connect_check_results.connect_monitor_id</code> . Test query performance on each index. Update CI to verify index presence.	MODERATE	HIGH
No code style enforcement (linting/formatting)	1. Add <code>.eslintrc.json</code> + <code>.prettierrc.json</code> to frontend (inherit from standard React config); add pre-commit hook or npm run format script. 2. Add <code>.php-cs-fixer.php</code> to backend; run in CI after <code>phpunit</code> . 3. Update <code>ci.yml</code> to fail if lint errors found. This is blocking for agent integration.	MODERATE	CRITICAL
Minimal test coverage (only 2 unit tests)	1. Write integration tests for Discovery API (GET/POST <code>discovery_jobs</code> , tree traversal). 2. Write integration tests for Connect API (monitor CRUD, reachability checks). 3. Add API request/response validation tests. 4. Add React component tests for Discovery & Connect modules. Target: 60% backend coverage, 50% frontend coverage.	MODERATE	HIGH
No root CLAUDE.md (unified AI runbook)	Create <code>.discovery-src/CLAUDE.md</code> documenting: (1) stack & conventions (no <code>extract()</code> in PHP, functional React components only, manual SQL schema), (2) module structure (Discovery vs Connect, shared vs isolated tables), (3) test requirements (PHPUnit for backend, React Testing Library for frontend), (4) deployment workflow (manual SQL + docker-compose restart), (5) AI-safe refactors (e.g., "extract service layer," "split large components," "add query methods to models").	MODERATE	HIGH
No code repository branch protection	Add <code>.github/CODEOWNERS</code> assigning backend (<code>backend/</code>) to PHP team, frontend (<code>frontend/</code>) to React team. Add <code>.github/pull_request_template.md</code> with checklist requiring: tests pass, types checked (frontend), manual QA. Enable branch protection on main: require PR review (1 approver), require CI to pass, dismiss stale reviews.	MODERATE	MEDIUM

No structured logging / observability	1. Add Monolog to Laravel config (<code>config/logging.php</code>); ship logs to stderr (Docker-standard). 2. Integrate Sentry or Rollbar for error tracking (add client ID to <code>.env.example</code>). 3. Add APM instrumentation (Datadog or New Relic SDK) to measure API latency & throughput. 4. Add Prometheus metrics exporter (optional but recommended for scale).	HIGH RISK	MEDIUM
Manual SQL migrations (no framework versioning)	Migrate to Laravel migrations framework: 1. Convert <code>docker/mariadb/init.sql</code> to <code>database/migrations/2024_xx_xx_create_tables.php</code> . 2. Add <code>database/migrations/2024_xx_xx_create_indexes.php</code> for indexes. 3. Update CI to run <code>php artisan migrate</code> on test database before tests. 4. Document rollback procedure (e.g., <code>php artisan migrate:rollback</code>).	MODERATE	MEDIUM
PHPStan static analysis declared but not enforced	1. Create <code>phpstan.neon.dist</code> with level 5 (default); add Docker service for analysis. 2. Add <code>php artisan phpstan</code> (or <code>vendor/bin/phpstan analyze app/</code>) to CI pipeline. 3. Run before merge to catch type errors early.	MODERATE	LOW
No pre-commit hooks	Add Husky (npm) or pre-commit framework: run linters + formatters + type checks before commit. Prevent unstyle'd code from reaching CI.	MODERATE	LOW

8.9 Expected Outcomes

- **Phase 1 (Code Quality Gates):** Linting & formatting enforced in CI; agent-generated code passes style gates immediately.
- **Phase 2 (Test Baseline):** 60%+ backend test coverage; API endpoints verified before agent changes land; regressions caught early.
- **Phase 3 (AI Setup):** CLAUDE.md + module contracts define safe refactors; agents understand repo structure, constraints, and deployment workflow.
- **Phase 4 (DB Optimization):** Query performance is measurable; agents can confidently refactor queries without fear of regression; migration framework enables safe schema versioning.
- **Phase 5 (Observability):** Production issues are logged & traceable; agents can verify their changes in production; monitoring detects performance regressions in real-time.
- **Overall:** Readiness upgrades from **Moderate** to **Good**, enabling safe agent-driven refactoring, code generation, and automated testing across Discovery & Connect modules.